

DevOps Interview

Complete Q&A Reference — Linux | CI/CD | Kubernetes | AWS | Terraform

1. Linux & Package Management

Q: Walk me through your day as a DevOps Engineer.

Start with a standup — review overnight alerts, check pipeline health, pending PRs. Morning: infra changes, Terraform runs, reviewing monitoring dashboards (Datadog/Grafana). Afternoon: deployments, reviewing Kubernetes cluster health, triaging issues. End of day: review logs, write runbooks if any incidents occurred. Always context-switching between infra, CI/CD, and support tasks.

Q: What is a package manager?

A tool that automates installation, upgrade, configuration, and removal of software packages. It resolves dependencies automatically and maintains a local database of installed packages.

Q: What are the types of package managers?

- System-level (OS): apt/apt-get (Debian/Ubuntu), yum/dnf (RHEL/CentOS), pacman (Arch)
- Language-level: pip (Python), npm/yarn (Node.js), gem (Ruby), cargo (Rust), maven/gradle (Java)
- Container/infra: Helm (Kubernetes charts), Homebrew (macOS)

Q: What functionalities can we achieve using a package manager?

- Install, remove, update packages
- Automatic dependency resolution
- Version pinning and locking
- Repository management (add/remove repos)
- List installed packages and search available ones

Q: Can we upgrade or downgrade a particular CLI version? How?

Yes. Examples:

```
# apt – install specific version sudo apt install nginx=1.18.0-0ubuntu1 # pip – downgrade  
pip install ansible==2.9.27 # npm npm install -g npm@8.0.0
```

Q: What is the difference between sudo and su?

su (switch user): switches to another user account entirely, requires that user's password. Opens a new shell as that user.

sudo (superuser do): runs a single command with elevated privileges, requires the current user's own password. Granular control via /etc/sudoers. Preferred in production — audited and scoped.

Q: How to provide a user sudo-level privileges?

```
# Method 1 – add to sudo group usermod -aG sudo username # Debian/Ubuntu usermod -aG wheel  
username # RHEL/CentOS # Method 2 – edit sudoers (visudo) username ALL=(ALL:ALL) ALL #  
Method 3 – specific commands only (safer) username ALL=(ALL) NOPASSWD: /bin/systemctl  
restart nginx
```

Q: Once we execute a sudo command, what happens in the background?

- Shell looks up /etc/sudoers (via visudo) to check if user has permission
- PAM (Pluggable Authentication Modules) authenticates the user via their own password
- A child process is forked with elevated UID (0 = root)
- The command executes in that elevated context
- Event is logged to /var/log/auth.log (Ubuntu) or /var/log/secure (RHEL)

- Timestamp is cached for ~15 minutes — next sudo won't prompt again within window

Q: Have you worked on patching servers?

Yes. In the internship, we patched Linux servers using Ansible playbooks to keep them consistent. Process: snapshot the instance first (backup), run apt update && apt upgrade in dry-run, test on staging, then apply to prod during a maintenance window. For AMI patching — build a new AMI from a golden AMI baseline using Packer or AWS Systems Manager Patch Manager, test, then rotate EC2s via autoscaling.

2. CI/CD & Jenkins

Q: What are the CI/CD tools you have worked with?

Jenkins (primary — declarative pipelines, shared libraries), GitHub Actions (YAML-based workflows, PR-triggered pipelines). Familiar with GitLab CI concepts.

Q: What are the different types of pipelines?

- **CI Pipeline:** triggered on code push — build, test, scan
- **CD Pipeline:** triggered after CI — deploy to staging/prod
- **Declarative Pipeline:** structured Jenkins DSL using 'pipeline {}' block
- **Scripted Pipeline:** Groovy-based, more flexible but complex
- **Multibranch Pipeline:** auto-detects branches, creates pipelines per branch

Q: Write the stages in a Jenkins pipeline.

```
pipeline { agent any stages { stage('Checkout') { steps { git 'https://github.com/repo.git' } } stage('Build') { steps { sh 'mvn clean package -DskipTests' } } stage('Unit Test') { steps { sh 'mvn test' } } stage('SonarQube') { steps { sh 'mvn sonar:sonar' } } stage('Docker Build') { steps { sh 'docker build -t myapp:$BUILD_NUMBER .' } } stage('Trivy Scan') { steps { sh 'trivy image myapp:$BUILD_NUMBER' } } stage('Push to ECR') { steps { sh 'docker push $ECR_REPO/myapp:$BUILD_NUMBER' } } stage('Deploy') { steps { sh 'kubectl apply -f k8s/' } } } post { always { cleanWs() } failure { mail to:'team@company.com', subject:'Build Failed' } } }
```

Q: Advantages of Jenkins?

- Open-source with 1800+ plugins
- Distributed builds via master-agent architecture
- Supports parallel execution
- Deep integration with Docker, Kubernetes, AWS, GitHub
- Declarative pipelines stored as code (Jenkinsfile in repo)
- RBAC, audit logs, fine-grained access control

Q: What are parallel stages?

Running multiple stages simultaneously to reduce pipeline time. Example: run unit tests and security scans at the same time instead of sequentially.

```
stage('Parallel Checks') { parallel { stage('Unit Test') { steps { sh 'mvn test' } } stage('Lint') { steps { sh 'eslint src/' } } stage('Trivy Scan') { steps { sh 'trivy image myapp:latest' } } } }
```

Q: What are distributed executions?

Jenkins Master-Agent architecture. The master schedules jobs and agents execute them. You can have multiple agents (VMs, containers, cloud instances) running builds in parallel, distributing load. Useful when builds are resource-intensive or environment-specific (e.g., one agent for Java builds, another for Docker builds).

Q: If a pipeline is getting slower over time, how will you troubleshoot?

- Check Jenkins build logs — identify which stage is slow

- Enable timestamps plugin to pinpoint bottlenecks
- Check if Docker build cache is being invalidated (restructure Dockerfile)
- Check if Maven/npm downloads are happening every run — use a cache/artifactory
- Check if agents are overloaded — spread load across more agents
- Check if SonarQube/Trivy scans are timing out — tune thresholds
- Review parallel opportunities — convert sequential stages to parallel
- Check for workspace accumulation — add cleanWs() in post steps

3. Kubernetes

Q: What are taints and tolerations?

Taint: applied to a node — prevents pods from being scheduled on it UNLESS the pod has a matching toleration. Syntax: key=value:effect (NoSchedule | PreferNoSchedule | NoExecute).

Toleration: applied to a pod — allows (but doesn't force) scheduling on a tainted node.

```
# Add taint to node kubectl taint nodes node1 env=prod:NoSchedule # Pod toleration (in spec) tolerations: - key: 'env' operator: 'Equal' value: 'prod' effect: 'NoSchedule'
```

Use case: dedicate nodes for specific workloads (GPU nodes, prod-only nodes).

Q: If the scheduler has stopped working, which component brings it up?

The **kubelet** on the control plane node. The scheduler (kube-scheduler) runs as a **static pod** — its manifest lives in /etc/kubernetes/manifests/. The kubelet watches this directory and automatically restarts any static pod that crashes. The controller manager doesn't manage the scheduler — kubelet does.

Q: What if the Kube API server goes down?

- kubectl commands stop working — no new API requests can be processed
- **Existing running pods continue to run** — kubelet manages them independently
- No new pods can be scheduled (scheduler can't talk to API server)
- Controller manager stops reconciling (can't read/write cluster state)
- etcd is still running but nothing can read from it
- Recovery: SSH directly to control plane node, check /var/log/pods/ or journalctl -u kubelet, fix the static pod manifest in /etc/kubernetes/manifests/kube-apiserver.yaml

4. AWS

Q: What AWS services have you worked on?

EC2, VPC (subnets, route tables, IGW, NAT Gateway, security groups, NACLs), EKS, S3, IAM, RDS (MySQL, cross-region read replicas), ECR, CloudFront, Route53, Lambda, EBS, ALB/NLB, CloudWatch, Systems Manager.

Q: EC2 is deployed but not reachable. How to troubleshoot?

1. Check Security Group — inbound rule for port 22 (SSH) or 80/443 (HTTP/S)
2. Check NACL (Network ACL) — both inbound AND outbound rules
3. Check route table — does the subnet have a route to the IGW (public) or NAT (private)?
4. Verify instance state — is it 'running'? Check status checks (System + Instance)
5. Check if the instance has a public IP or Elastic IP (if public subnet)
6. Check OS-level firewall — iptables/ufw inside the instance
7. Check the application — is the service actually running? (systemctl status)
8. Use VPC Flow Logs to see if traffic is being rejected

Q: EC2 is in a private subnet with a public IP, still can't reach the internet. Why?

Root cause: A private subnet has no route to the Internet Gateway (IGW) in its route table. A public IP alone means nothing — internet traffic must have a route. Even with a public IP, if the route table says 0.0.0.0/0 → local only, packets have nowhere to go.

Fix: Use a NAT Gateway (deployed in a public subnet). Update the private subnet's route table: 0.0.0.0/0 → NAT Gateway. This allows outbound internet access without making the instance directly reachable from the internet.

Q: How can I attach a public IP to an EC2 in a private subnet?

Technically you can assign an Elastic IP to it, but it still won't have internet access because the route table for a private subnet has no IGW route. A public IP is only meaningful in a public subnet. For outbound internet from private subnet, use NAT Gateway. For inbound access, use a Load Balancer in the public subnet that forwards to private EC2s, or a bastion host.

Q: What is an EBS volume?

Elastic Block Store — persistent block-level storage that attaches to an EC2 instance. Think of it as a virtual hard drive. Persists independently of EC2 instance lifecycle. Types: gp3 (general purpose, default), io2 (high IOPS for databases), st1 (throughput-optimized HDD), sc1 (cold HDD). EBS volumes are AZ-specific.

Q: Reduce EBS volume from 300GB to 200GB. How?

Critical fact: You CANNOT shrink an EBS volume. AWS only allows expansion, not reduction.

Correct approach:

- 1. Create a new 200GB EBS volume in the same AZ
- 2. Attach it to the EC2 instance
- 3. Format and mount the new volume
- 4. Copy data: `rsync -avz /old-mount/ /new-mount/`
- 5. Update `/etc/fstab` to point to new volume
- 6. Unmount old volume, detach it
- 7. Snapshot old volume before deleting (safety)

Q: Options available to copy data into an S3 bucket?

- AWS Console — manual upload via browser
- AWS CLI: `aws s3 cp` / `aws s3 sync`
- AWS SDK (boto3 for Python, Java SDK, etc.)
- S3 Transfer Acceleration — faster transfers over long distances
- AWS DataSync — large-scale on-prem to S3 migration
- AWS Snowball/Snowball Edge — physical device for petabyte-scale transfers
- S3 Batch Operations — bulk operations on existing objects
- Cross-region/cross-account replication — automated replication rules
- Multipart Upload API — large files split into parallel parts

Q: Have you worked on patching AMIs?

Yes. Approach: start from a base/golden AMI, run Ansible playbook or AWS Systems Manager Run Command to apply security patches (`apt upgrade` / `yum update`), then create a new AMI using `CreateImage` API or Packer. The new AMI gets validated, then launch templates are updated and EC2s are rotated via Auto Scaling Group instance refresh. Old AMIs are deregistered after a retention period.

Q: What are the properties of load balancers?

- **ALB (Application Load Balancer):** Layer 7, path/host-based routing, WebSocket, HTTP/2, target groups, best for microservices
- **NLB (Network Load Balancer):** Layer 4, ultra-high performance, static IP, TLS termination, best for TCP/UDP workloads

- **CLB (Classic):** Legacy, Layer 4+7, avoid in new architectures
- **Common:** Health checks, cross-zone load balancing, SSL/TLS termination, access logs, sticky sessions

Q: How to add an additional EBS volume to EC2?

1. Create EBS volume in same AZ as EC2 via Console or CLI
2. Attach: `aws ec2 attach-volume --volume-id vol-xxx --instance-id i-xxx --device /dev/xvdf`
3. SSH into EC2, run `lsblk` to confirm device appears
4. Format: `mkfs.ext4 /dev/xvdf`
5. Create mount point: `mkdir /data`
6. Mount: `mount /dev/xvdf /data`
7. Add to `/etc/fstab` for persistence across reboots

Q: How to replicate an RDS instance from one region to another?

Use **Cross-Region Read Replicas**. Steps:

- Enable automated backups on the source RDS instance (required)
- Create a read replica and specify a different region
- AWS handles replication via async log shipping (for MySQL/PostgreSQL)
- For DR failover: promote the read replica to standalone DB in the target region
- Use Route53 latency/failover routing to switch traffic

Note: Read replicas have replication lag. For zero RPO, use Aurora Global Database instead.

Q: What are the S3 storage classes?

- **S3 Standard:** frequent access, low latency, highest cost
- **S3 Intelligent-Tiering:** auto-moves objects between tiers based on access patterns
- **S3 Standard-IA (Infrequent Access):** lower cost, retrieval fee, min 30-day storage
- **S3 One Zone-IA:** same as IA but single AZ — cheaper, less durable
- **S3 Glacier Instant Retrieval:** archival, milliseconds retrieval, quarterly access
- **S3 Glacier Flexible Retrieval:** minutes to hours retrieval
- **S3 Glacier Deep Archive:** cheapest, 12-hour retrieval, long-term archival

5. Terraform

Q: 2 people create an EC2 at the same time using Terraform in the same AWS account. What happens?

With remote backend + DynamoDB locking (correct setup): The second person's terraform apply is blocked with a lock error until the first completes. State file is locked in DynamoDB.

Without state locking (local state or no DynamoDB): Both operations run simultaneously, causing state file corruption — one person's changes may overwrite the other's, leading to drift between actual infra and state.

```
# Correct backend config with locking terraform { backend "s3" { bucket = "my-tf-state" key
= "prod/terraform.tfstate" region = "us-east-1" dynamodb_table = "terraform-lock" # <--
this is the lock encrypt = true } }
```

Q: Create EC2 instances in 2 different AWS accounts using Terraform. How?

Use **multiple provider blocks with aliases** and `assume_role` for cross-account access.

```
provider "aws" { alias = "account_a" region = "us-east-1" assume_role { role_arn =
"arn:aws:iam::111111111111:role/TerraformRole" } } provider "aws" { alias = "account_b"
region = "ap-south-1" assume_role { role_arn =
"arn:aws:iam::222222222222:role/TerraformRole" } } resource "aws_instance" "ec2_a" {
provider = aws.account_a ami = "ami-0abcdef1234567890" instance_type = "t3.micro" }
resource "aws_instance" "ec2_b" { provider = aws.account_b ami = "ami-0fedcba9876543210"
instance_type = "t3.micro" }
```

Prerequisite: Each account must have an IAM role with a trust policy allowing the calling identity to assume it.

Prepared for Sanket Ajay Chopade — DevOps Engineer Interview Reference

Sections: Linux | CI/CD | Kubernetes | AWS | Terraform